Universe and Tools

HarmonyGames · Fixed environment date: February 14, 2026, 00:00 UTC · Updated September 26, 2026

What the simulated company is, which thirteen services it runs on, how the clock works, and what “deep enough” means before you start building.

Change log

Date	Updates
Sep 26, 2026	Rewritten for Spec V2. Confluence and Snowflake confirmed in scope and flagged for regeneration. Baseline model updated to Opus 5.5. Taxonomy screenshots removed. Golden-solution and hint references removed throughout.
Aug 2026	Original release.

1 · The company

HarmonyGames is a founder-led mobile game studio. It is small enough that one person's Slack DMs matter and large enough that nobody has the whole picture. That combination is what makes it useful as an evaluation universe: the information an agent needs genuinely exists, and it is genuinely scattered.

Fact	Detail
Flagship title	A domino-matcher, the studio's established revenue source
Newer title	A zombie match-3, still finding its audience
Third title	A fighting game, most recently launched
Staff personas	19, across product, engineering, design, data, marketing, finance and operations
Services	13 connected systems
Environment date	February 14, 2026, 00:00 UTC — a Saturday

2 · The thirteen services

Service	What lives there	What it is good for
Slack	Channels, DMs, the running commentary on everything	The richest source of evidence and of distractors. Most tasks start here
Gmail	External correspondence, publishers, investors, vendors	The outside view, which frequently disagrees with the inside one
Google Drive	Shared files and folder structure	Where a written deliverable is expected to land

Service	What lives there	What it is good for
Google Docs	Design docs, specs, legal documents, post-mortems	Long-form statements of intent that the data can contradict
Google Sheets	Tracking sheets, budgets, financial records	Numbers that must reconcile against Snowflake and against each other
Google Slides	Pitch decks, quarterly reviews, reports	Summarised figures that go stale the moment the data moves
Google Calendar	Meetings, availability, scheduling	Ordering evidence, and a natural place for a deadline to live
Contacts	People lookup, email addresses, roles	Resolving “tell whoever owns it” into a real recipient
GitHub	Code, pull requests, reviews, release history	The ground truth about what actually shipped and when
Linear	Engineering tickets, bugs, projects, cycles	Both a source and a destination — most tasks end in a ticket
Trello	Roadmap, feature and prototype boards	Product-side planning, often out of date relative to Linear
Confluence	Team wiki and knowledge base	Policies, conventions and process documents an agent should apply
Snowflake	Data warehouse, analytics tables	The only place many figures can be derived rather than read

> **UNIVERSE WORK IN FLIGHT**
>
> Engineering is regenerating the synthetic Confluence and Snowflake data so it matches the real Drive and Excel data, and is clearing personal information out of the artifacts. Both services remain fully in scope for a task. If you recorded a Confluence page or a Snowflake row count before the refresh lands, **re-verify it afterwards** — values may move, and a rubric asserting a stale figure becomes an incorrect criterion.

3 · The clock

The universe is frozen at **February 14, 2026, 00:00 UTC**. Everything before that instant is the past. Nothing after it has happened.

Rule	Why it matters
Write dates absolutely	“February 20, 2026”, never “next week” or “by Friday”. Relative dates break on replay and are scored as errors
Nothing may post-date the clock	A prompt or a seeded record dated after February 14, 2026 describes something that has not happened. Failing error
Created before updated	A record whose `created_at` is later than its `updated_at`, or a comment that predates its parent, is a temporal field inconsistency
Working hours are real hours	Business messages at three in the morning, or a weekday standup landing on a Sunday, read as synthetic
February 14 is a Saturday	Check the weekday before you seed anything recurring

4 · What the agent can and cannot see

Your assigned persona controls what the agent is allowed to **read**. Four services are shared by everyone; the rest are filtered.

Class	Services	Behaviour
Unscoped — shared	GitHub · Trello · Linear · Contacts	Every persona sees the same records
Scoped — persona-filtered	Gmail · Slack · Google Calendar · Drive · Docs · Sheets · Slides	`list`, `search` and by-ID reads all filter. A denied read returns “not found”
Not classified	Confluence · Snowflake	Neither column names them. Open each one from the persona's view and confirm the record is readable before you depend on it

> **TWO RULES THAT CATCH PEOPLE OUT**
>
> **Writes are never governed by ACL.** Determine what your persona can write from the tool catalog, never from what it can read.
>
> **Universe Explorer is author god-mode.** It shows you everything. The agent and the verifier both use the persona's scoped view. Never build a task on, or grade against, evidence only you can see.

The full policy, including the failure signatures of each ACL mistake, is in the **Persona ACL Policy** guideline.

5 · The depth bar

“Investigate something and send one email” is not a task. Before you write a prompt, check it against all four of these.

Requirement	What fails it
Real tool use	A question general knowledge can answer without touching the universe
Genuinely cross-service	One service resolves the whole thing, or a second is touched only trivially. Failing error
Investigation and action	A pure lookup, or a pure write with nothing to work out first
Not pre-solved	The prompt states the root cause, leaving the agent nothing to diagnose. Failing error

Three or more services is the working target. Two is the floor, and only when the second one carries real weight.

6 · Making the universe carry the difficulty

You add depth by **seeding data** — adding or editing universe records so that the work lives in what the agent has to discover, not in how the prompt is worded.

> TERMINOLOGY
>
> Call it **data seeding** or **injection**. "SQL injection" means a security vulnerability and is not what this is.

The six consistency rules

Rule	What it means	Failure signature if you break it
On the clock	Every date sits before February 14, 2026, and created never post-dates updated	Temporal field inconsistency
Consistent across services	Anything referenced somewhere exists where it is referenced	Cross-service referential inconsistency — a Slack thread about a PR that is not in GitHub
Coherent with the universe	Figures reconcile with the records they are derived from	Quantitative reconciliation inconsistency — a doc saying $65 against a sheet saying $35
Structurally valid	No invented fields; IDs follow the existing format	Identifier inconsistency — an ID that breaks the `ENG-####` or `ZOM-###` pattern
Reachable and findable	Your persona can read it, and a realistic search path reaches it	Unsolvable task, or evidence nobody can find
Natural, not pre-solving	Seed the symptom and the evidence, never the conclusion	Pre-solved prompt, or AI-flavoured boilerplate in the seeded text

State and lifecycle

Two further defects are easy to miss because nothing about them looks wrong in isolation.

- **State inconsistency.** A ticket marked Closed whose linked pull request was never merged; a task marked Done with an open blocker still attached.
- **Semantic inconsistency.** The same programme described under two different names in two services, with nothing connecting them. This one is sometimes deliberate — if you intend the agent to reconcile the two names, say so in your Oracle Events. If you did not intend it, it is a defect.

7 · Your injection SQL is the record

The SQL you submit is the only record of what you changed. A reviewer re-injects it into a fresh environment when a re-run is needed.

- If it does not reproduce your universe exactly, **your task cannot be re-run at all**, and it will be returned.
- Every statement must be valid against the real schema. Invented columns fail.
- Keep the changelog entry aligned with the SQL — a changelog describing an edit the SQL does not make is its own defect.
- Run `/injection` in the eval sandbox immediately after you write it. It checks structure, references, IDs, dates and coherence, and catches most of the above before it costs you a cycle.

8 · Tool behaviour worth knowing before you build

Behaviour	Consequence for your design
Search is fuzzy; fetch is exact	If your prompt names the exact document or thread, the agent fetches it directly and skips discovery entirely
Async waits affect search only	The matching fetch stays instant. An async wait only bites when discovery is genuinely required
A forced error looks like an ACL denial	On a scoped service the agent cannot tell them apart. Do not force an error on a tool whose denial the agent is meant to reason about
Snowflake answers must be derived	Figures there usually need a query and an aggregation, which is exactly why Snowflake criteria tend to carry high weights
Blocked and repeated calls still count	They add nothing to real depth, and an inflated tool-call count is a failing error in its own right

9 · Before you leave the universe step

- Every date in every seeded record sits before February 14, 2026.
- Every cross-reference resolves to a record that actually exists.
- Every figure reconciles with the record it is derived from, unless the disagreement is the point of the task.
- Every ID follows the existing format for its service.

- Your persona can read every piece of evidence the task depends on — confirmed from the persona's view, not from Universe Explorer.
- At least three services carry real weight.
- Nothing you seeded states the conclusion the agent is supposed to reach.
- `/injection` returned PASS.

Original Conference · Universe and Tools · Spec V2 · Updated Sep 26, 2026